

Feedback Granularity for LLM Repair in Intent-to-Configuration NetOps Tasks: A Preregistered Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired experiment to test whether localized, actionable verifier feedback reduces LLM repair effort versus a global pass/fail signal when generating network configurations from intent. Using a frozen preregistration (cnsmair-2026-feedback-granularity-v1), we synthesized 120 intent-to-configuration tasks (paired within-task), ran a single initial generation and one repair attempt per condition with gpt-5-mini, and verified outputs with an emulator+Batfish verifier. Confirmatory result: both conditions passed verification on all 120 tasks (baseline = 120/120, guarded/localized = 120/120), producing a paired success-rate difference of 0.00 (exact McNemar $p = 1.00$; 95% bootstrap CI [0.00, 0.00]). Secondary/exploratory subgroup analyses were not pre-registered as confirmatory and remain exploratory. We report complete preregistration, execution artifacts, and analysis manifests; we interpret the null confirmatory outcome, discuss limitations (single model, constrained task bank, emulator-based verification), and provide a Disclosure Statement detailing autonomy, models, prompts, and artifacts.

I. INTRODUCTION

Automated intent-to-configuration systems using large language models (LLMs) promise to simplify NetOps but raise operational safety and reliability concerns because LLM outputs can be syntactically plausible yet incorrect. Prior work has emphasized verifier-integrated generate-verify-repair loops and localized verification feedback to guide repairs (e.g., Mondal et al. 2023; Batfish evolution). We preregistered the research question: does localized, actionable verifier feedback reduce repair iterations and total model-call cost compared to a global pass/fail signal in emulator-verified intent-to-configuration tasks? We followed a frozen preregistration (study id cnsmair-2026-feedback-granularity-v1) and executed the paired experiment described below.

II. RELATED WORK

Closest prior work demonstrates substantial benefits from interposing verifiers and locality-aware feedback in LLM-driven config tasks and GVR-style pipelines (Mondal et al., 2023; Batfish tooling and configuration-verification literature). Work on LLM-based configuration validation and LLM-backed generate-verify-repair architectures motivated our design (Lian et al., ICSE 2025; Cadenas, SSRN 2026). These sources motivated a paired, verifier-grounded, operationally realistic evaluation focusing on repair-effort endpoints rather than unconstrained generation benchmarks.

III. METHODOLOGY

Preregistration: we preregistered the full design (cnsmair-2026-feedback-granularity-v1). Design: paired within-task comparison (baseline global pass/fail vs. guarded/localized feedback). Tasks: 120 sanitized intent-to-configuration tasks stratified across four topology/error-clusters; a pilot of 20 tasks validated templates (pilot reported but excluded from the confirmatory test). Model and prompts: hosted model calls to gpt-5-mini (model provider recorded as openai_responses in execution manifest). Execution: for each task we invoked one shared initial-generation call and then, per condition, allowed up to one repair call (preregistered stopping rules). Verification: emulator + Batfish-based verifier; verifier logs, localized failure payloads, model outputs, provider-call records, and hashes were archived (see execution manifest and artifact hashes). Primary estimand: paired mean difference in repair iterations required to reach verifier pass (Localized - Global). Primary test: paired Wilcoxon (pre-specified) and paired McNemar for the success-rate comparison; bootstrap CIs (10,000 replicates) reported. Contamination and exclusion rules, retry policy, and exact analysis recipes were frozen in the preregistration and followed during execution; all artifacts and hashes are recorded in the execution manifest.

IV. RESULTS

Execution and analysis were completed against the preregistered plan. Confirmatory outcome: all 120 included pairs achieved verifier pass in both conditions (baseline success = 120/120; guarded/localized success = 120/120). The paired success-rate difference (guarded - baseline) = 0.00. Exact McNemar two-sided $p = 1.00$. Paired bootstrap (10,000 replicates, seed recorded) 95% CI for the paired difference = [0.00, 0.00]. No pairs were deterministically excluded under the preregistered rules; no infrastructure failures occurred. Secondary pre-specified analyses (model-call counts, time-to-pass) were recorded but produced no between-condition variation in the confirmatory sample; exploratory subgroup tests were designated exploratory in the preregistration and are reported only descriptively in the artifacts.

V. DISCUSSION

Interpretation: in this controlled, preregistered paired study using gpt-5-mini and an emulator+Batfish verifier on a 120-task bank, providing localized actionable verifier feedback did

not improve the pass-rate or reduce repair-iteration counts versus a global pass/fail signal: both approaches achieved full verification coverage. Possible explanations include (1) task-bank design and difficulty level that allowed the model to succeed under both regimes after at most the preregistered repair attempt; (2) the single-model choice (gpt-5-mini) and prompt/formulation may have been sufficient for these tasks; or (3) the verifier-localized feedback, as instantiated, did not add discriminative information beyond the global signal for these task instances. This null confirmatory result is evidence against a large practical advantage of the specific guarded-feedback instantiation under these exact conditions, not evidence against localized feedback in other models, tasks, or verifier designs. Exploratory analyses remain limited and are reported as such.

VI. LIMITATIONS

- Single-model scope: experiment used only gpt-5-mini; results do not generalize across LLM families or sizes.
- Task-bank constraints: tasks were synthetic/sanitized operator-style templates limited to IPv4 reachability and ACL/policy constraints; broader NetOps tasks (e.g., routing dynamics, multi-vendor CLIs, stateful services) were not evaluated.
- Verification environment: verifier is emulator + Batfish; behavior may differ from production devices or richer execution contexts.
- Operation budget and stopping rules: each pair allowed only one repair attempt per condition per preregistration; different iteration budgets could change outcomes.
- Exploratory analyses limited: subgroup and failure-mode analyses were exploratory and not powered for confirmatory claims; those analyses are available in the artifacts for reanalysis.

VII. CONCLUSION

We present a fully preregistered, executed, and analyzed paired study examining feedback granularity for LLM repair in intent-to-configuration NetOps tasks. The preregistered confirmatory result was a null effect: both global and localized-feedback treatments reached 100% verifier pass on the 120 paired tasks. The result is narrowly bounded by the study design (single model, fixed prompts, task bank, and verifier). We release the full preregistration, execution artifacts, scoring outputs, and analysis manifests to support independent reanalysis and follow-up work.

DISCLOSURE STATEMENT

Mandatory disclosure (study `cnsmair-2026-feedback-granularity-v1`). LLM(s) used: gpt-5-mini (model identifier recorded in `execution/model_configuration.json`; provider recorded as `openai_responses`). Orchestration/framework: `hosted_netops_gvr_v1` adapter family implementing a generate-verify-repair loop (preregistered). Initial master prompt and frozen input bundle: "Master Prompt v1 — Autonomous CNSM 2026 Research Run" (frozen input

bundle supplied pre-lock; preregistration and execution adhered to the autonomy lock described in that bundle). Topic constraints set by the Research Director limited scope to Generative AI / LLMs for NetOps with emphasis on reliability, verification, and repair. Code/experiment generation framework: the autonomous pipeline generated all prompts, canonicalization code, task templates, and experiment harness; code and configs are archived at paths referenced in the execution manifest. Tools permitted: emulator + Batfish verifier for config verification; no external proprietary simulators beyond recorded provider LLM calls. Preregistration/freeze: full preregistration (`cnsmair-2026-feedback-granularity-v1`) was created and frozen before observing final experiment outcomes; preregistration file and SHA-256 are recorded in the execution manifest. Autonomous literature, experiment, and analysis procedures: literature review, gap selection, experiment design, execution, artifact collection, and analysis were performed autonomously after the autonomy lock as recorded. Manuscript-generation and reference-verification procedures: manuscript drafting, autonomous peer-review passes, and automated integrity checks were executed without human manual editing after the autonomy lock; all cited references were verified against primary sources using public APIs and stored in the verified-records list. Technical restarts/deviations: execution manifest records no unplanned technical restarts; model-call budget and retry policy (up to two infrastructure retries) were followed; no post-hoc changes to the scientific design were made after observing results. Pre-lock development: prior infrastructure development and rehearsal occurred pre-lock; none of those pre-lock experimental outputs were used as empirical evidence for this confirmatory study. Autonomous finalization: the autonomous system produced the final IEEE-format manuscript package and archived all artifacts, logs, and hashes (see `execution/artifact_hashes` in the execution manifest). No human manually edited the manuscript text after the autonomy lock.

REFERENCES

- [1] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194
- [2] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866
- [3] Xinyu Lian, Yinfang Chen, Runxiang Cheng, Jie Huang, Parth Thakkar, Minjia Zhang, Tianyin Xu, "Large Language Models as Configuration Validators", 2025, 10.1109/icse55347.2025.00017
- [4] <https://doi.org/10.2139/ssrn.6754899>
- [5] Divya Raghunathan, Ryan Beckett, Aarti Gupta, David Walker, "ACORN: Network Control Plane Abstraction using Route Nondeterminism", 2022, 10.48550/arxiv.2206.02100